```
    .style("left", function(d,i,j) {
         return (Math.abs(i - 4) * 100) + "px";
       });
};

function restoreColumns() {
  d3.selectAll("div.datarow")
     .selectAll("div.data")
     .transition()
     .duration(2000)
     .style("left", function(d,i,j) {
            return (i * 100) + "px";
     });
};
```

There you have it—a sortable animated spreadsheet that, if you catch it in midtransition as I have in figure 8.5, is rather messy. It's animated, interactive, and data-driven with no SVG at all. Rather than adding more interactivity to our spreadsheet, we'll switch gears and focus on a second kind of traditional component of a web page: image galleries. But before we get to that, we'll need some images. Instead of loading them from external files, we'll draw our own PNGs using HTML5 canvas, an API made for drawing static images. We're not going to dive deep into canvas, but just use it to create circles with numbers on them to stand in for whatever images we might put in a gallery.

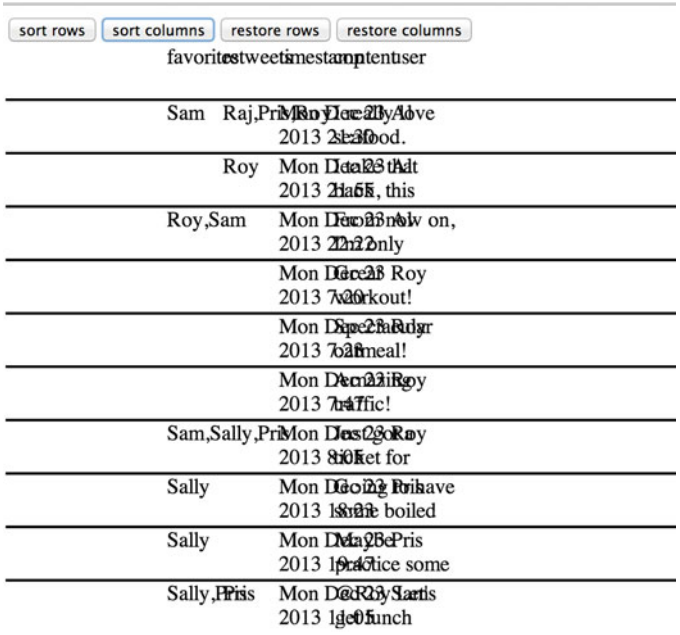


Figure 8.5 Sorting columns in our sheet. Because we didn't define a background value for the divs, the text floats over itself. In this screenshot, you can see that I've added all the buttons for sorting and restoring columns and divs.

8.3 Canvas

We won't use canvas too much here, but you should recognize that, although the canvas drawing syntax like that in listing 8.7 is different from SVG, it's something you